Introduction to Queue Data Structures:

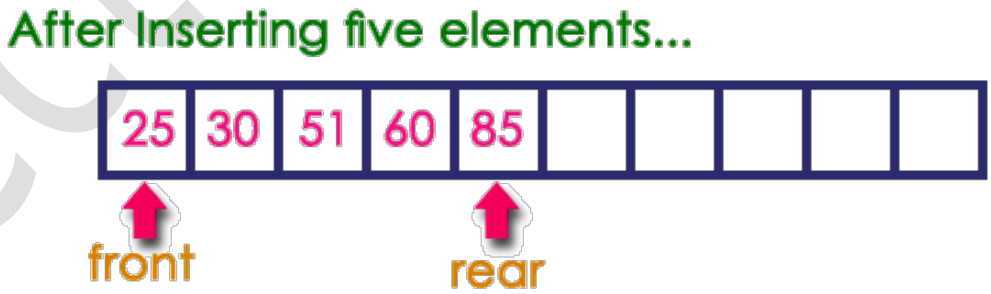
Queue is a linear data structure where the data is inserted from one end called Rear and the deleted from the other end called as Front.
Front points to the beginning of the Queue and rear points to the end of the queue.
In queue data structure, the insertion and deletion operations are performed based on **FIFO (First In First Out)** principle.
According to its FIFO structure , data inserted first will also be removed first.



In a queue data structure, the insertion operation is performed using a function called "**enQueue()**" and deletion operation is performed using a function called "**deQueue()**".

Example

Queue after inserting 25, 30, 51, 60 and 85.



Vector India

Operations on a Queue

The following operations are performed on a queue data structure...

1. **enQueue(value) - (To insert an element into the queue)**
2. **deQueue() - (To delete an element from the queue)**
3. **display() - (To display the elements of the queue)**

Queue data structure can be implemented in two ways. They are as follows...

1. **Using Array**
2. **Using Linked List**

When a queue is implemented using an array, that queue can organize an only limited number of elements. When a queue is implemented using a linked list, that queue can organize an unlimited number of elements.

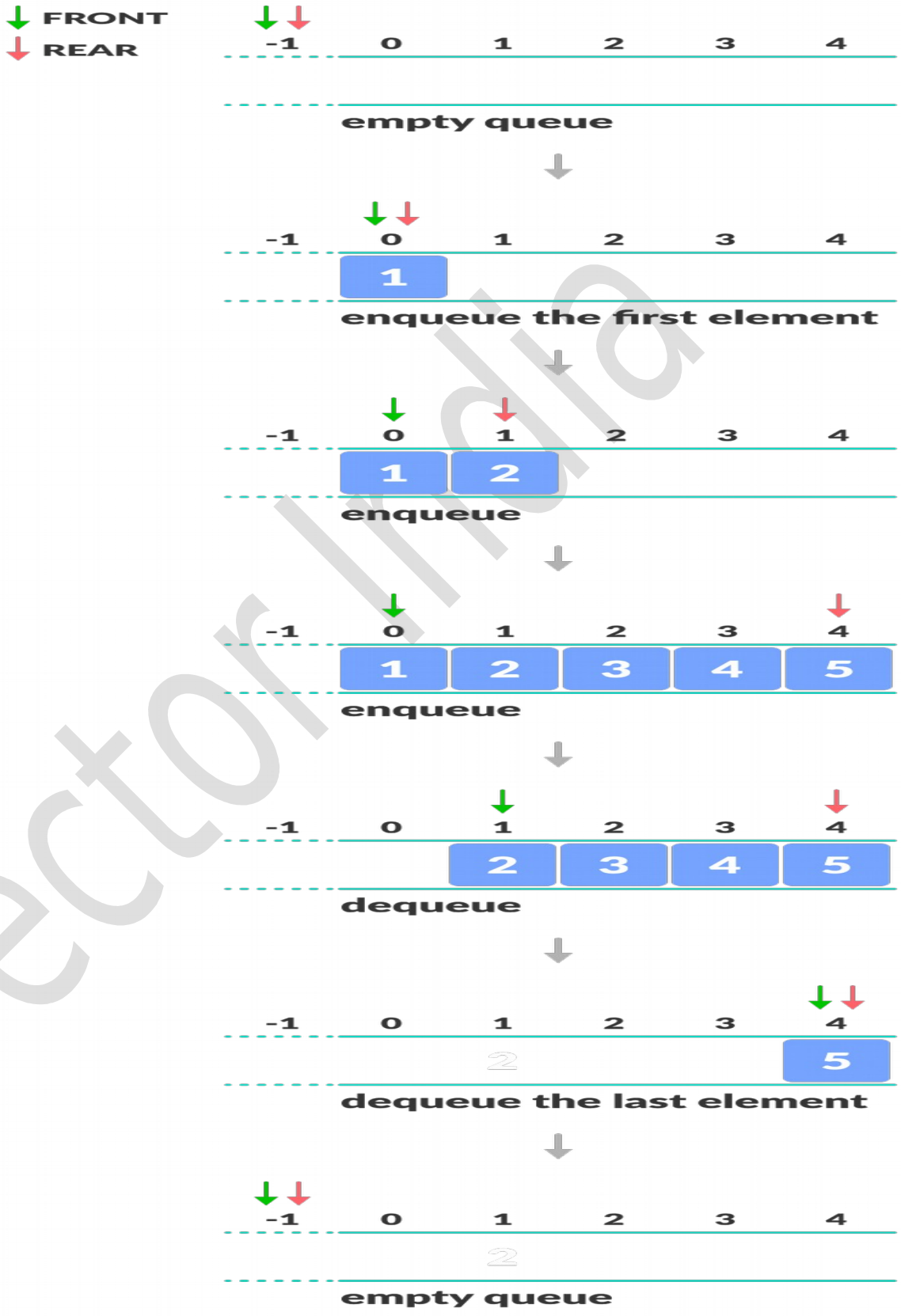
Algorithm for Enqueue Operation:

- check if the queue is full
- if the queue is full, produce overflow error and return
- if the queue is not full for the first element, set value of *FRONT* to 0
- increase the *REAR* index by 1
- add the new element in the position pointed to by *REAR*

Algorithm for Dequeue Operation

- check if the queue is empty
- If the queue is empty , produce the underflow error and return.
- If queue is not empty , return the value pointed by *FRONT* and increase the *FRONT* index by 1
- for the last element, reset the values of *FRONT* and *REAR* to -1

Vector India



Vector India

Applications of Queue

Queue, as the name suggests is used whenever we need to manage any group of
objects in an order in which the first one coming in, also gets out first while the others wait for their turn, like in the following scenarios:

- CPU scheduling, Disk Scheduling
- When data is transferred asynchronously between two processes.The queue is used for synchronization. eg: IO Buffers, pipes, file IO, etc
- Handling of interrupts in real-time systems.
- Call Center phone systems use Queues to hold people calling them in an order

Complexity Analysis of Queue Operations:

Just like Stack, in case of a Queue too, we know exactly, on which position new element will be added and from where an element will be removed, hence both these operations requires a single step.

Enqueue: O(1)

Dequeue: O(1)

Size: O(1)

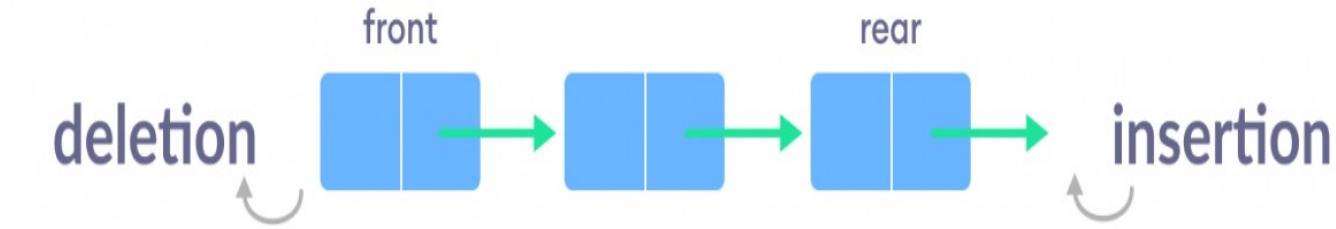
Types of Queue:

1. Simple Queue
2. Circular Queue
3. Priority Queue

Simple Queue

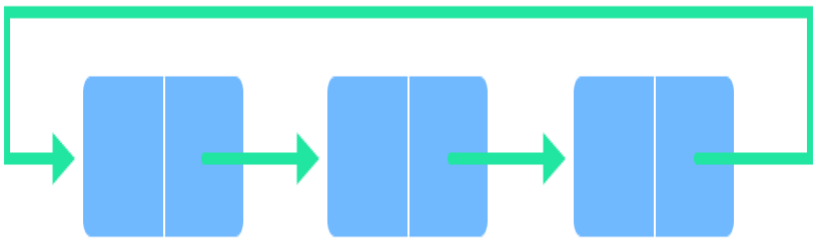
In a simple queue, insertion takes place at the rear and removal occurs at the front. It strictly follows FIFO rule.

Vector India



Circular Queue

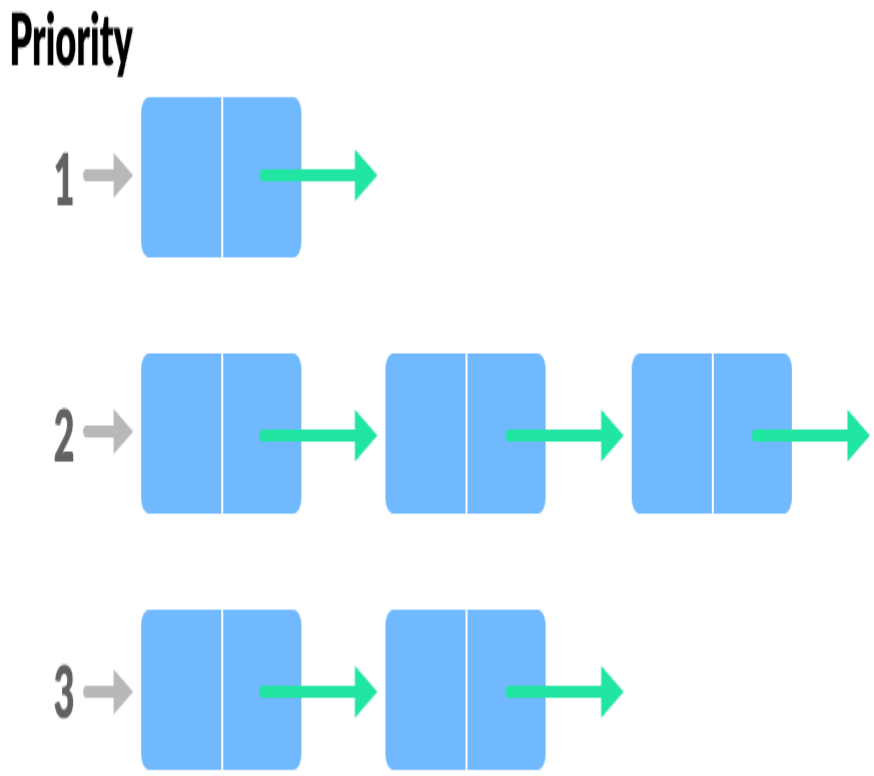
In a circular queue, the last element points to the first element making a circular link.



The main advantage of a circular queue over a simple queue is better memory utilization. If the last position is full and the first position is empty then, an element can be inserted in the first position. This action is not possible in a simple queue.

Priority Queue

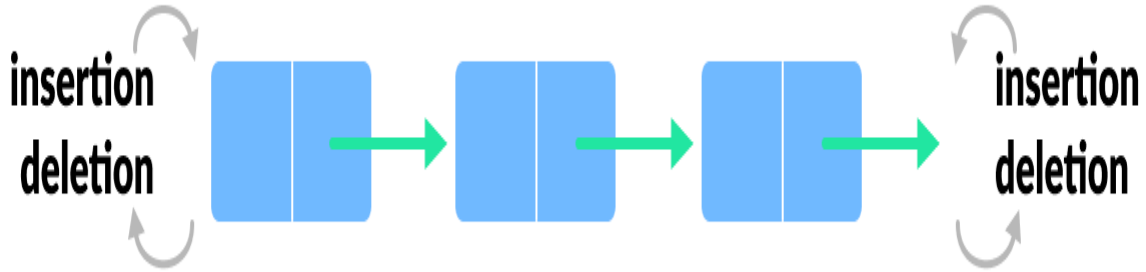
A priority queue is a special type of queue in which each element is associated with a priority and is served according to its priority. If elements with the same priority occur, they are served according to their order in the queue.



Insertion occurs based on the arrival of the values and removal occurs based on priority.

Deque (Double Ended Queue)

Double Ended Queue is a type of queue in which insertion and removal of elements can be performed from either from the front or rear. Thus, it does not follow FIFO rule (First In First Out).



Note :
Reference Books : Taken contents and diagrams from various websites.

Vector India